

Détection et classification de manoeuvres GEO par apprentissage supervisé : le dataset SPLID - draft

Etienne Chevrolat

Résumé

Ce document rend compte du travail effectué sur le dataset SPLID (MIT ArcLab) : recréation complète de la pipeline de données, réimplémentation from scratch en PyTorch de cinq architectures de détection de manoeuvres — dont une adaptation de l’architecture gagnante du *MIT ARCLab Prize for AI Innovation in Space 2024* — et mise en place de l’évaluation officielle du concours (score F_2 avec tolérance temporelle). Le meilleur modèle est un CNN purement convolutif à backbone dupliqué par direction (`cnn_split`), doté d’une fenêtre d’entrée asymétrique (128 pas passés, 32 futurs). Il atteint en validation un F_2 de 0,920 en détection de noeuds et 0,896 sur la chaîne complète détection + classification, la classification des noeuds étant quasi parfaite (99,5% sur le type de noeud, 99,0% sur le type de propulsion). Une analyse par direction montre que le gain provient presque entièrement de l’*allongement du contexte passé*, et non du changement d’architecture : à fenêtre égale, les architectures profondes sont indiscernables entre elles. Sur le jeu de test officiel du concours, en revanche, le score retombe à 0,724 / 0,675 : nous montrons que cet écart s’explique par un décalage de distribution entre les jeux d’entraînement et de test. Enfin, nous caractérisons quantitativement la raison de l’échec de l’inférence sur les TLE réels de SpaceTrack : SPLID fournit les *éléments osculateurs* de trajectoires *simulées*, échantillonnés régulièrement, là où les TLE sont des *éléments moyens* mesurés à cadence irrégulière — deux domaines de données incompatibles.

1 Le dataset SPLID et la tâche

1.1 Structure du dataset

Le dataset SPLID (*Satellite Pattern-of-Life Identification Dataset*) caractérise le Pattern of Life (PoL) de satellites GEO par une suite de noeuds (points de rupture) séparant des *behavioral modes* (modes de comportement constants entre deux noeuds). Nous utilisons le jeu `phase_2`, celui du concours : **1900 objets d’entraînement** et **501 objets de test**, chacun décrit par une série temporelle de 2172 pas de temps à cadence fixe de 2 h (soit environ 6 mois), contenant les éléments orbitaux osculateurs, les positions géographiques et l’état cartésien du satellite.

Labels SPLID

- **Noeuds** : SS (début de période d'étude), ID (*Initiate Drift*), AD (*Adjust Drift*), IK (*Initiate station-Keeping*), ES (fin de période d'étude) ;
- **Types (modes)** : NK (pas de maintien à poste), CK / EK / HK (maintien à poste chimique / électrique / hybride) ;
- **Directions** : chaque noeud est labellisé Est-Ouest (EW, manoeuvres *in plane*) et/ou Nord-Sud (NS, manoeuvres *out of plane*).

Le jeu d'entraînement contient **5360 noeuds de manoeuvre** (2169 IK, 1812 ID, 1379 AD) et 3800 noeuds SS.

Remarque importante. Ce jeu `phase_2` est à distinguer de l'échantillon réduit livré avec le `splid-devkit` (250 objets, 373 noeuds de manoeuvre seulement). L'écart de volume est déterminant : à architecture et hyperparamètres identiques, le passage de l'échantillon réduit au jeu complet fait passer le F_2 de validation de 0,59 à 0,89. La quantité de labels était ici le facteur limitant, bien avant le choix d'architecture.

1.2 La métrique officielle du concours

Le concours évalue une soumission (liste de noeuds prédits par objet, avec direction, type de noeud et type de propulsion) par appariement aux noeuds vrais avec une **tolérance de ± 6 pas de temps** (12 h), par objet et par direction. Un noeud apparié est un TP si son type de noeud *et* son type de propulsion sont corrects, un FP sinon ; les prédictions non appariées sont des FP et les noeuds vrais manqués des FN. Le score agrégé est le

$$F_2 = \frac{5 TP}{5 TP + 4 FN + FP}$$

qui favorise le rappel (il vaut mieux sur-détecter que manquer une manoeuvre), complété par le RMSE des écarts temporels des TP. Nous avons réimplémenté cet évaluateur à l'identique du `splid-devkit` (`src/ml/splid_eval.py`), avec deux modes :

- **détection seule** : restreint aux noeuds de manoeuvre (ID/AD/IK), l'appariement temporel suffit — mesure la qualité du localizer ;
- **chaîne complète** : mode concours, tous les noeuds, un TP exigeant que le type de noeud et le type de propulsion soient corrects.

2 Recréation de la pipeline de données

Toute la pipeline, des fichiers CSV bruts aux tenseurs PyTorch, a été recrée dans `src/ml/` (`datahandler.py`, `targets.py`, `dataset.py`), configurée par Hydra (`configs/ml/`).

Pipeline localizer (détection)

1. **Features continus** : les éléments angulaires discontinus ($e, i, \Omega, \omega, \nu$) sont convertis en éléments équinoxiaux continus ($k, h, q, p, \cos \lambda, \sin \lambda$), ce qui évite les sauts $360^\circ \rightarrow 0^\circ$;
2. **Dérivées discrètes** : on ajoute les incréments $\Delta a, \Delta q, \Delta p$ entre pas consécutifs, qui portent la signature impulsionnelle des manoeuvres, soit 9 features au total ;
3. **Normalisation** : standardisation ajustée sur le train uniquement ;
4. **Cibles** : deux canaux (EW, NS) valant des bosses triangulaires de demi-largeur 6 pas centrées sur les noeuds de manoeuvre, à zéro ailleurs ;
5. **Fenêtrage** : chaque exemple est une fenêtre glissante de 97 pas (48 passés, 48 futurs) centrée sur l’instant à prédire, soit un tenseur (9, 97) ;
6. **Split** : 80/20 *par objet* (1520 objets train, 380 objets validation), pour éviter toute fuite entre train et validation.

Le **classifieur** réutilise les mêmes features et fenêtres, mais échantillonnées uniquement aux instants des noeuds labellisés ; il prédit deux sorties par fenêtre : le type de noeud (ID/AD/IK) et le type de propulsion (NK/CK/EK/HK), avec une pondération de classes compensant le déséquilibre.

Corrections apportées à la pipeline existante. Plusieurs défauts ont été identifiés et corrigés, dont trois bloquants :

- le *scheduler* de learning rate (warmup linéaire puis décroissance cosine) n’était mis à jour que pour la tâche de pré-entraînement. Pour le localizer et le classifieur, le learning rate restait donc figé à sa valeur de départ, nulle : **les modèles n’apprenaient rien** (loss constante, $F_2 = 0$) ;
- le meilleur checkpoint était sélectionné sur la *val loss*. Or celle-ci diverge par surapprentissage alors que les métriques de détection continuent de progresser : pour le classifieur, le checkpoint retenu était celui de l’époque 0 (20,9% d’exactitude) au lieu de l’époque 58 (93,0%). Le critère est désormais le F_2 (localizer) et le F1 macro moyen (classifieur) ;
- les jeux de features SPLID et SpaceTrack étaient mélangés (la feature temporelle dt , propre aux TLE irréguliers, et les unités du demi-grand axe) ; ils sont maintenant explicitement séparés.

S’y ajoutent deux corrections de moindre visibilité mais aux conséquences directes sur ce rapport :

- la pondération de classes du classifieur était calculée sur le mauvais champ des échantillons (l’indice du *type de noeud* au lieu de celui du *type de propulsion*) et, surtout, n’était jamais transmise à la fonction de coût. Le déséquilibre des régimes de propulsion — la classe EK est la plus rare — n’était donc pas compensé ;
- les dimensions des couches denses étaient codées en dur (par exemple 64×35 en sortie du LSTM), valeurs valables pour la seule fenêtre de 97 pas. Elles sont désormais calculées à partir de la géométrie des convolutions. Sans cette correction, l’expérience décisive de ce rapport — le passage à une fenêtre de 161 pas — aurait été impossible sans réécriture manuelle des modèles.

3 Les modèles, recréés from scratch

Cinq architectures de localizer ont été réimplémentées et comparées, à features identiques :

- **naive_baseline** : une simple couche linéaire sur la fenêtre aplatie (calibre le problème) ;
- **small_cnn** : 3 couches de convolution 1D (64, 64, 48 canaux, noyaux 7) + tête dense ;
- **small_lstm** : une couche LSTM (32 unités) + max-pooling temporel + tête dense ;
- **cnn_lstm1** : **CNN et LSTM en série** : les 3 couches de convolution extraient les motifs locaux et compresse l'axe temporel ($97 \rightarrow 40$), puis un LSTM (64 unités) agrège séquentiellement, suivi d'un max-pooling et d'une tête dense à 2 sorties (probabilité d'un noeud EW / NS au centre de la fenêtre) ;
- **cnn_split** : **CNN pur à backbone dupliqué par direction**, décrit ci-dessous.

L'architecture réellement retenue par le gagnant. La lecture du dépôt de la solution gagnante (`localizer_sweeper.py`) montre que sa configuration finale de localizer ne comporte **aucune couche récurrente** (`lstm_layers: []`), contrairement à ce que suggère le nom usuel « CNN-LSTM ». Elle repose sur deux caractéristiques que **cnn_lstm1** ne reproduisait pas :

- **split_cnn: True** : deux piles de convolutions **entièrement indépendantes** traitent la même fenêtre d'entrée, l'une dédiée aux noeuds EW, l'autre aux noeuds NS. L'intuition est que les manoeuvres in-plane (signature sur a) et out-of-plane (signature sur i) ont des signatures trop différentes pour un backbone commun, qui imposerait un compromis pénalisant la direction la moins représentée ;
- une fenêtre d'entrée **asymétrique** : 128 pas passés pour seulement 32 futurs, là où nous utilisons 48/48.

Notre **cnn_split** reprend ces deux idées, avec une pile plus profonde que celle du gagnant (5 blocs au lieu de 3, avec de la dilatation pour élargir le champ réceptif à moindre coût). Chaque bloc suit l'ordre du gagnant, Conv1d $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Dropout, la normalisation intervenant *après* l'activation. Les deux branches sont ensuite concaténées vers une tête dense partagée (64, 32), chaque direction conservant sa propre sortie. L'implémentation d'origine est en Keras ; elle a été réécrite en PyTorch (`src/ml/model.py`).

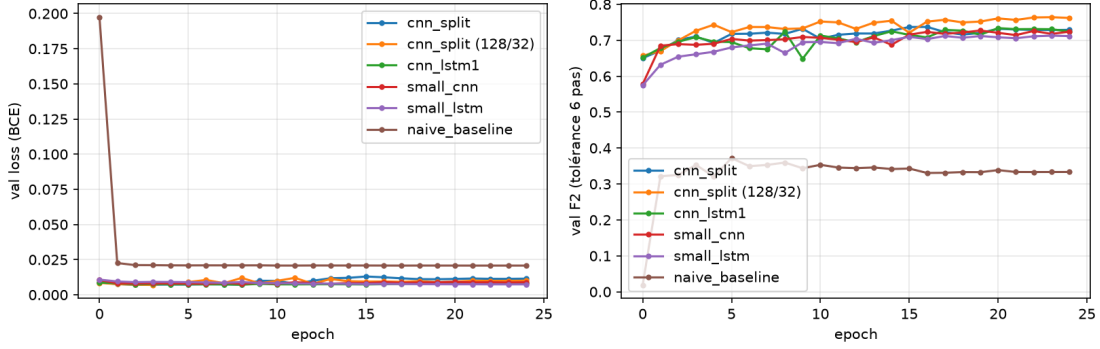
L'entraînement minimise une *binary cross-entropy* sur les deux canaux, avec AdamW, warmup linéaire puis décroissance cosinus ($3 \cdot 10^{-4}$, batch 256, 25 epochs). Le backbone **cnn_lstm1** avec une tête double (cross-entropy) sert de classifieur. Les entraînements ont été menés sur GPU (NVIDIA RTX 4000 Ada).

4 Résultats

4.1 Comparaison des architectures

Courbes de validation des localizers (loss BCE et F_2 à seuil de détection fixe de 0,1) :

Localizer : courbes de validation par modèle



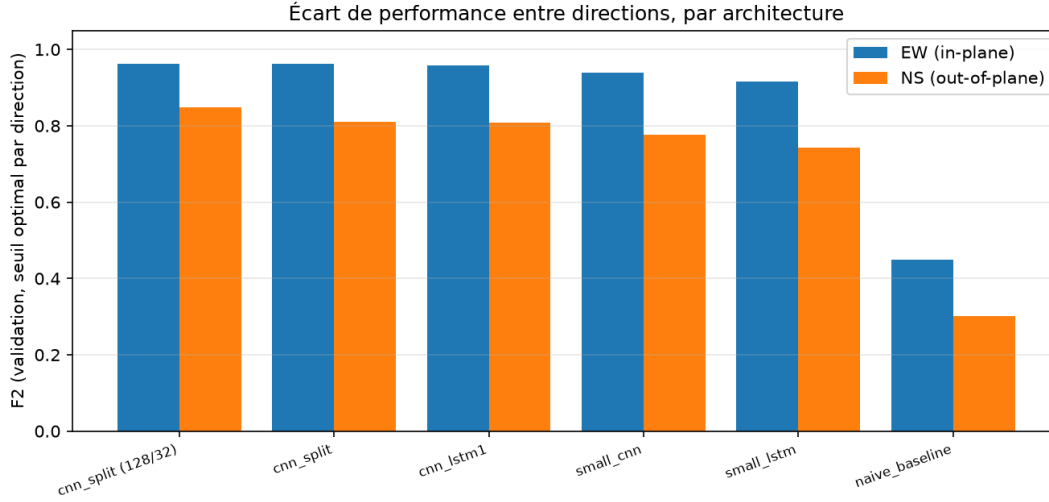
Modèle	Fenêtre	F_2 max en validation (seuil 0,1)
naive_baseline	48/48	0,372
small_lstm	48/48	0,713
small_cnn	48/48	0,727
cnn_lstm1 (série)	48/48	0,734
cnn_split	48/48	0,741 et 0,737
cnn_split	128/32	0,764

La baseline linéaire décroche nettement, ce qui confirme que la tâche exige bien une modélisation de la structure temporelle. Les architectures profondes, en revanche, se tiennent dans un mouchoir de poche à fenêtre égale (0,713 à 0,741).

Mesurer le bruit avant d’interpréter un écart. Un lancement accidentel en double de `cnn_split` (48/48) a fourni une estimation gratuite de la variabilité run-à-run : deux exécutions strictement identiques donnent 0,741 et 0,737, soit un écart de 0,004. Or l’avance de `cnn_split` sur `cnn_lstm1` à fenêtre égale est de 0,007 — du même ordre. **À fenêtre égale, le passage au CNN pur à backbone dupliqué n’apporte donc pas de gain mesurable.** Seul l’allongement de la fenêtre produit un écart franc (+0,023), bien au-delà du bruit.

4.2 Où se situe réellement la difficulté : l’écart entre directions

Le score global masque une asymétrie forte entre les deux directions. Nous mesurons donc chaque architecture direction par direction, chacune à son propre seuil optimal, afin que la comparaison ne dépende pas d’un seuil global favorisant la direction dominante :



Modèle	Fenêtre	F_2 EW (in-plane)	F_2 NS (out-of-plane)
naive_baseline	48/48	0,450	0,302
small_lstm	48/48	0,916	0,742
small_cnn	48/48	0,940	0,776
cnn_lstm1	48/48	0,958	0,808
cnn_split	48/48	0,963	0,810
cnn_split	128/32	0,962	0,849

Trois enseignements se dégagent, et ils recadrent le diagnostic initial :

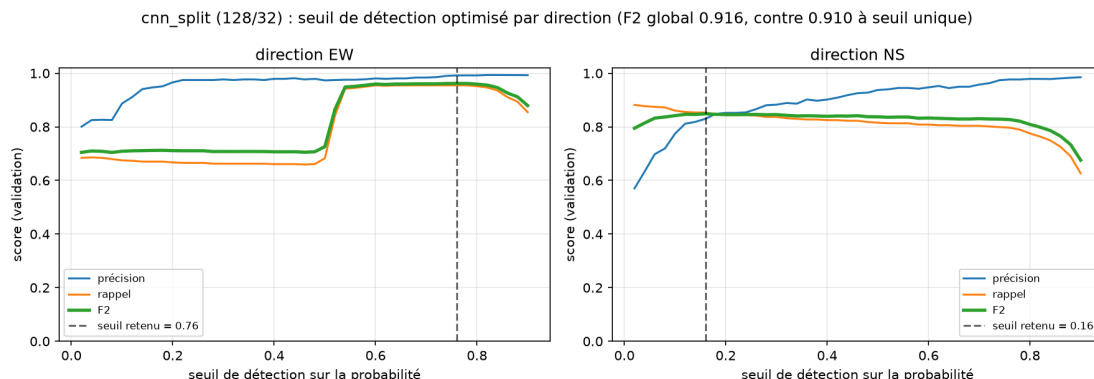
La détection NS est systématiquement le maillon faible. Quelle que soit l'architecture, le F_2 NS accuse 0,12 à 0,17 de retard sur le F_2 EW. Le déficit porte sur le *rappel* (les manoeuvres out-of-plane sont manquées, non sur-détectées), et non sur la précision.

Le backbone dupliqué ne corrige pas cet écart. À fenêtre égale, `cnn_split` obtient 0,810 en NS contre 0,808 pour `cnn_lstm1` : la séparation des deux branches, qui devait précisément éviter qu'un backbone commun sacrifie la direction minoritaire, ne change rien de mesurable. L'hypothèse d'un compromis imposé par le partage de paramètres n'est donc pas ce qui limitait la détection NS.

C'est le contexte passé qui compte. Tout le gain NS provient de l'allongement de la fenêtre (0,810 $\rightarrow$ 0,849, rappel 0,799 $\rightarrow$ 0,853), à architecture inchangée. Ce résultat est cohérent avec la physique du problème : une manoeuvre in-plane produit une discontinuité nette et locale du demi-grand axe, lisible dans une fenêtre courte, tandis qu'une manoeuvre out-of-plane se manifeste comme une *rupture de pente* de l'inclinaison, qui ne devient identifiable qu'avec plusieurs jours de recul. Passer de 4 à 10,7 jours de contexte passé donne au réseau de quoi estimer la pente de part et d'autre du noeud.

4.3 Choix du seuil de détection

La sortie du localizer est une probabilité par pas de temps ; les noeuds prédits sont les centres des plages au-dessus d'un seuil. Ce seuil arbitre précision contre rappel. Les comptages TP/FP/FN s'additionnant entre directions, nous les tabulons une fois par direction et par seuil, puis cherchons le couple (seuil EW, seuil NS) optimal sur la grille :



Le seuil optimal est très différent d'une direction à l'autre : 0,76 en EW contre 0,16 en NS. Cet écart est une mesure directe de l'asymétrie décrite plus haut — le réseau est franchement confiant sur les noeuds EW, où il produit des pics de probabilité nets, et beaucoup plus timide sur les noeuds NS, dont les probabilités doivent être acceptées bien plus bas pour ne pas sacrifier le rappel. Optimiser les deux seuils séparément porte le F_2 global à 0,916, contre 0,910 au meilleur seuil unique : le gain est réel mais modeste, l'essentiel étant que le réglage par direction évite d'imposer à NS un seuil calibré sur EW.

Relever le seuil au-delà du défaut de 0,1 apporte en revanche un gain net (F_2 passant de 0,764 à 0,916) : le modèle produit des pics francs, et remonter le seuil élimine beaucoup de fausses alarmes de faible amplitude sans coûter de rappel.

C'est une différence de fond avec les méthodes par seuillage de résidus (filtre de Kalman, cf. rapport précédent) : le seuil s'applique ici à une probabilité apprise, homogène d'un satellite à l'autre, et non à une grandeur physique dont l'échelle varie selon l'objet. Il se règle donc *une seule fois* pour tout le catalogue, ce qui lève l'obstacle principal à l'automatisation identifié précédemment.

4.4 Score officiel en validation

Évaluateur officiel du concours sur les 380 objets de validation :

	Précision	Rappel	F_2	RMSE (pas)
Détection seule (ID/AD/IK)	0,945	0,914	0,920	0,54
Chaîne complète (mode concours)	0,761	0,938	0,896	0,41

En détection seule : 972 TP pour seulement 57 FP et 91 FN. Le RMSE de 0,54 pas signifie que les noeuds détectés sont localisés à moins de 1,1 h de la manoeuvre vraie en moyenne — une précision temporelle très supérieure à la tolérance de 12 h.

4.5 Score sur le jeu de test officiel

Le seuil ayant été calibré sur la validation, nous l’appliquons tel quel aux 501 objets de test officiels, jamais vus :

	Précision	Rappel	F_2	RMSE (pas)
Détection seule (ID/AD/IK)	0,901	0,690	0,724	0,49
Chaîne complète (mode concours)	0,423	0,793	0,675	0,33

À comparer honnêtement au gagnant du concours, qui obtient $F_2 = 0,805$ sur ce même jeu de test en chaîne complète : nous sommes en deçà (0,675). L’écart entre nos scores de validation (0,896) et de test (0,675) est le résultat le plus instructif de cette étude, et mérite d’être décomposé.

Le rappel chute, pas la précision. En détection seule, la précision se maintient (0,901 contre 0,945) mais le rappel s’effondre (0,690 contre 0,914) : le modèle ne produit pas davantage de fausses alarmes, il rate simplement des manoeuvres qui ne ressemblent pas à celles vues à l’entraînement.

Un décalage de distribution des labels. La distribution des types de propulsion diffère fortement entre les deux jeux :

Type de propulsion (noeuds SS)	NK	CK	EK	HK
Entraînement (3800 noeuds)	55 %	22 %	19 %	4 %
Test (1000 noeuds)	26 %	27 %	32 %	15 %

Le jeu de test est donc bien plus riche en propulsion électrique (EK) et hybride (HK) — précisément les régimes à poussée faible et prolongée, dont la signature est la plus ténue. Cela explique à la fois la perte de rappel du localizer et l’effondrement de la précision en chaîne complète : sur 1516 noeuds vrais du test, 737 sont correctement localisés dans le temps mais reçoivent un type erroné, et comptent alors comme faux positifs. Notre classifieur, entraîné sur une distribution dominée par NK, se généralise mal à cette répartition.

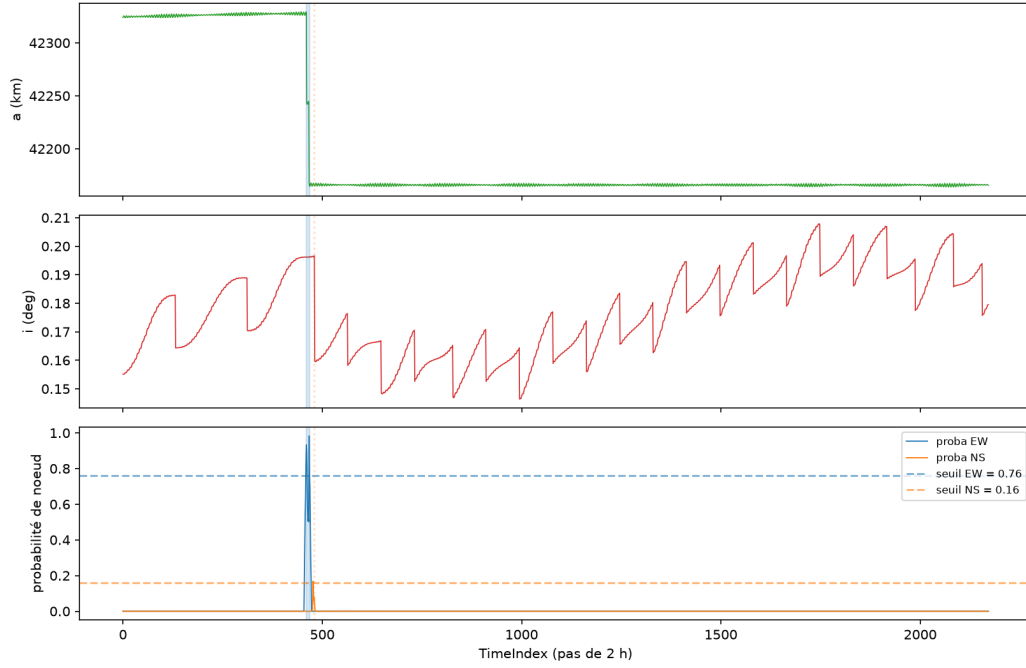
Une limite de notre construction de soumission. S’y ajoute un choix discutable de notre part : nous prédisons systématiquement les deux noeuds SS à l’instant $t = 0$, dont la fenêtre d’entrée est à moitié constituée de *padding*. Le type de propulsion y est donc estimé dans les pires conditions, alors que ces noeuds pèsent pour les deux tiers des noeuds du jeu de test. Une tête dédiée, exploitant une fenêtre uniquement postérieure à $t = 0$, est la première amélioration à apporter.

4.6 Exemples qualitatifs

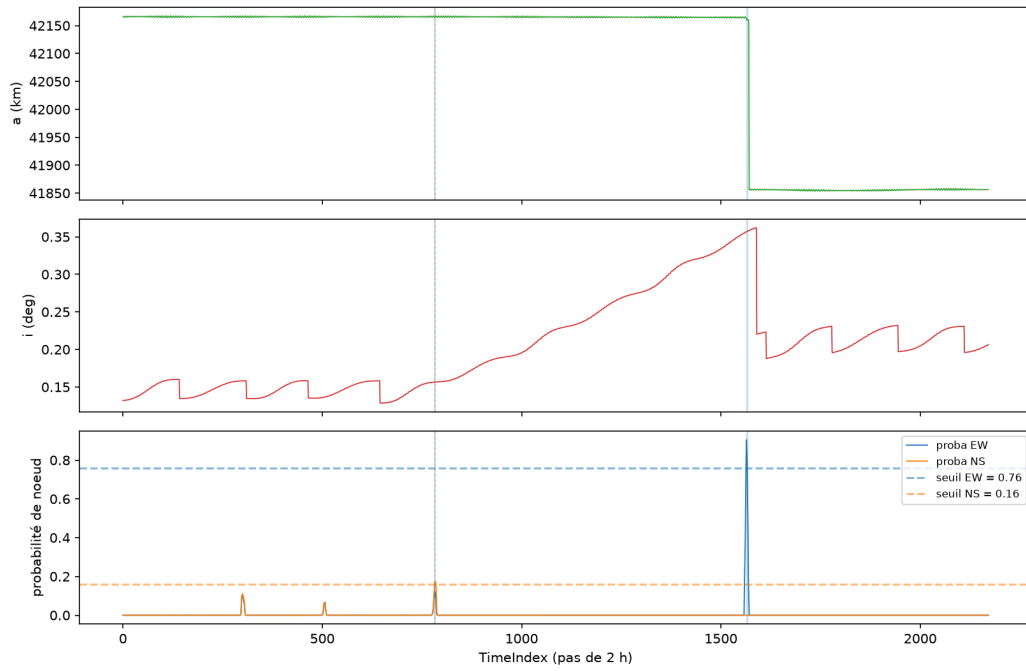
Échantillon aléatoire (tirage déterministe) de dix objets de validation comportant au moins trois noeuds de manoeuvre. Pour chacun : demi-grand axe, inclinaison, et sortie du localizer, superposés aux labels (traits verticaux, EW en bleu plein, NS en orange pointillé). Le titre indique les TP / FP / FN de l’objet.

Les labels SPLID ne marquent pas toutes les manoeuvres, mais uniquement les changements de *pattern of life*.

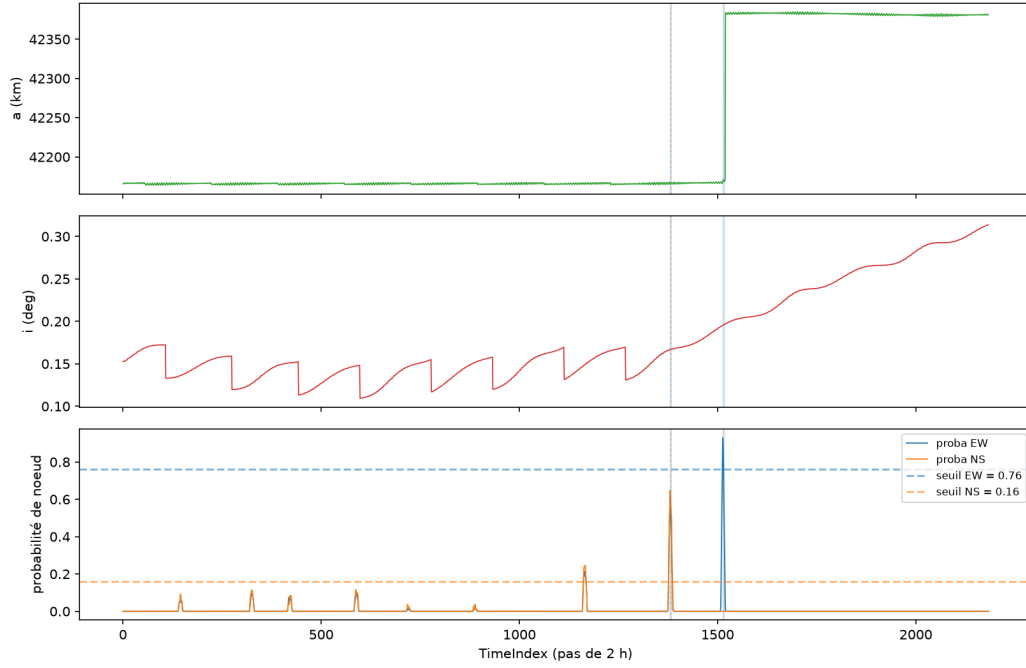
Objet 117 (validation) : labels (traits verticaux) vs sortie du localizer — TP 3 / FP 0 / FN 0



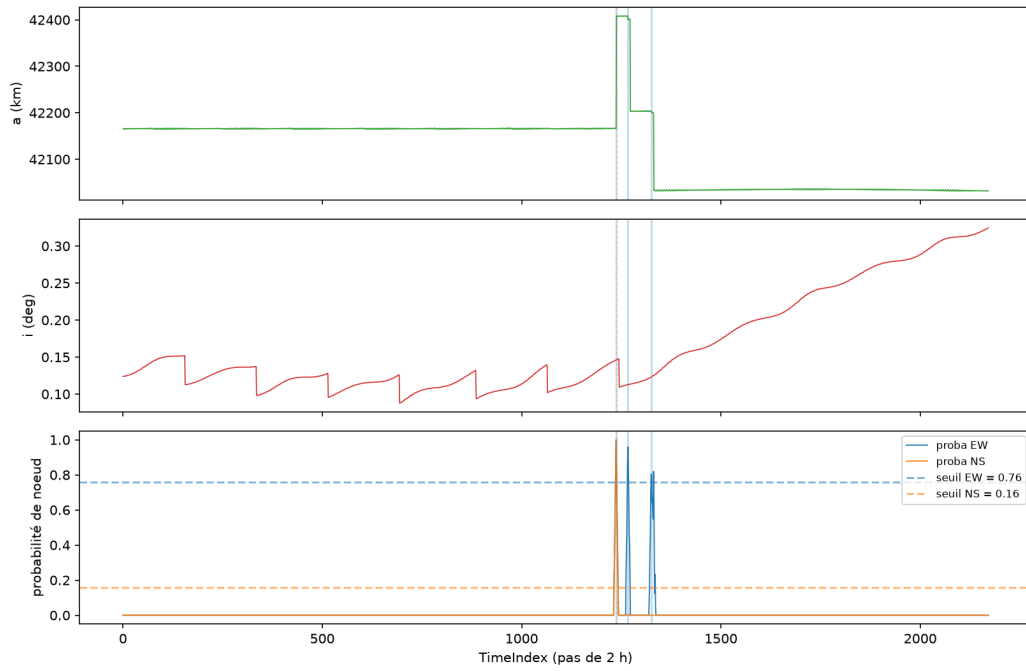
Objet 144 (validation) : labels (traits verticaux) vs sortie du localizer — TP 2 / FP 1 / FN 1



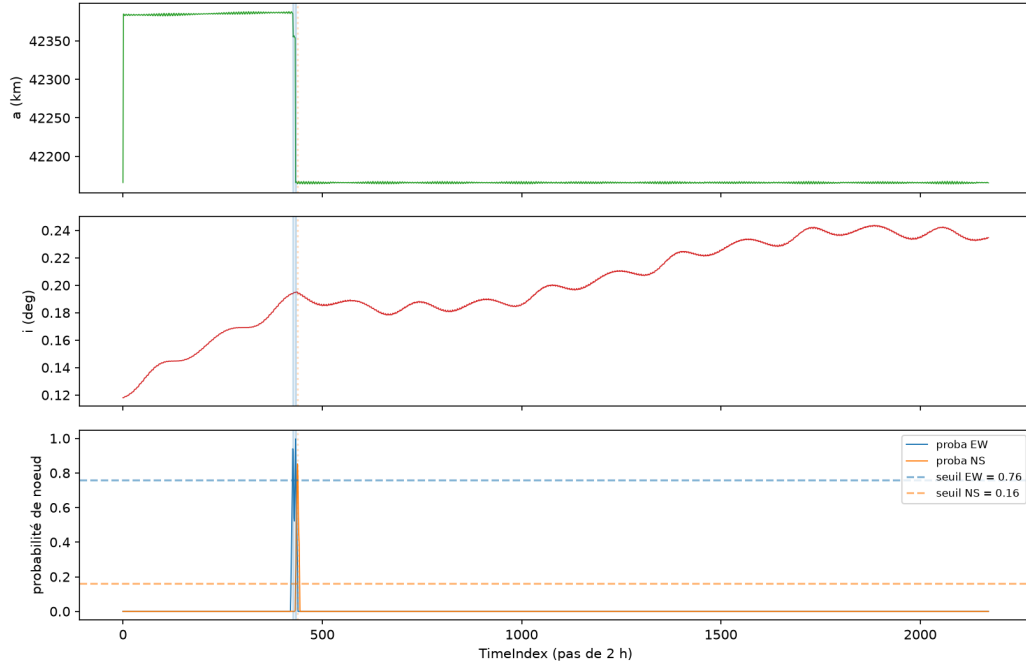
Objet 209 (validation) : labels (traits verticaux) vs sortie du localizer — TP 2 / FP 1 / FN 1



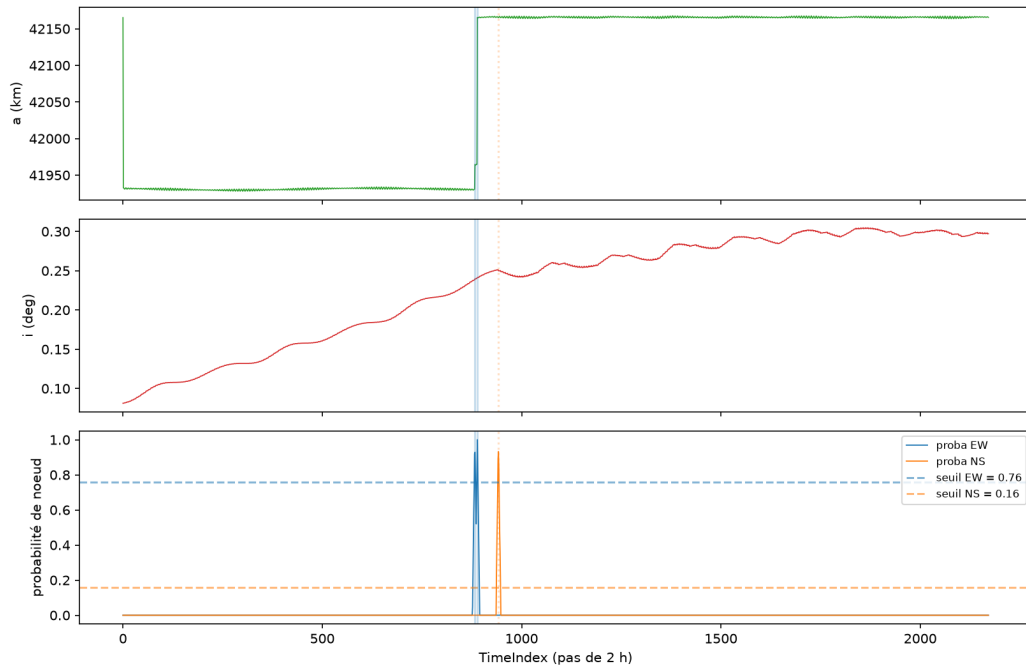
Objet 391 (validation) : labels (traits verticaux) vs sortie du localizer — TP 4 / FP 1 / FN 0



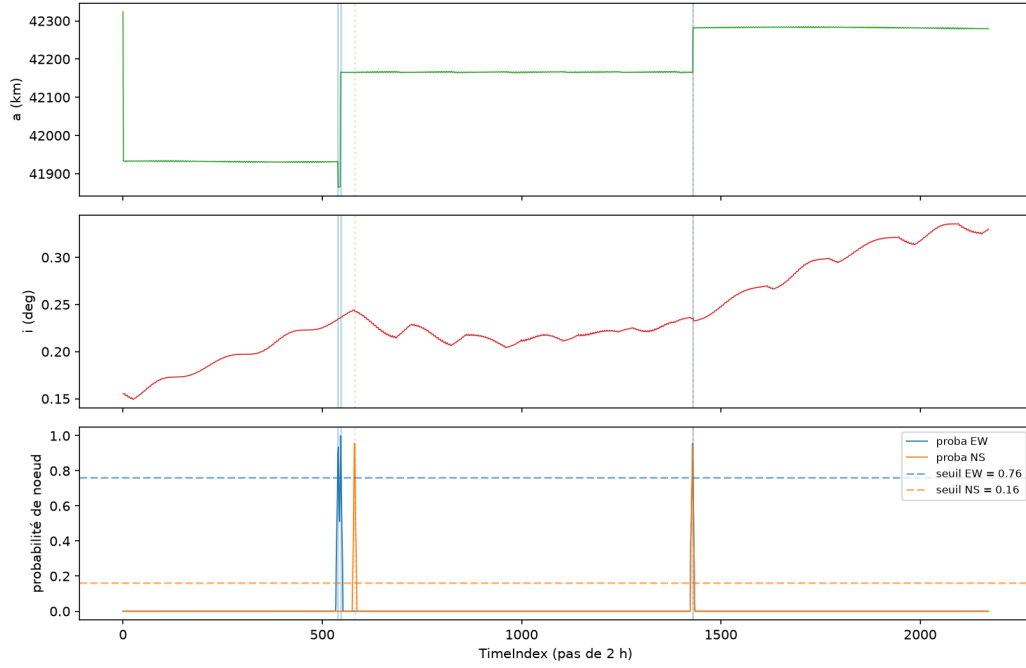
Objet 542 (validation) : labels (traits verticaux) vs sortie du localizer — TP 3 / FP 0 / FN 0



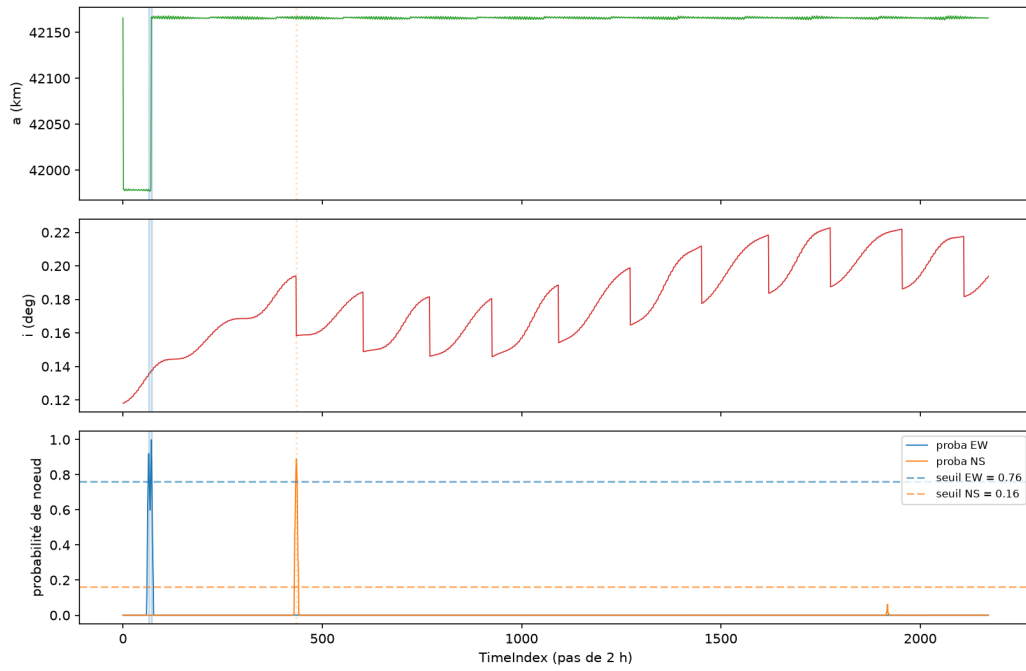
Objet 573 (validation) : labels (traits verticaux) vs sortie du localizer — TP 3 / FP 0 / FN 0

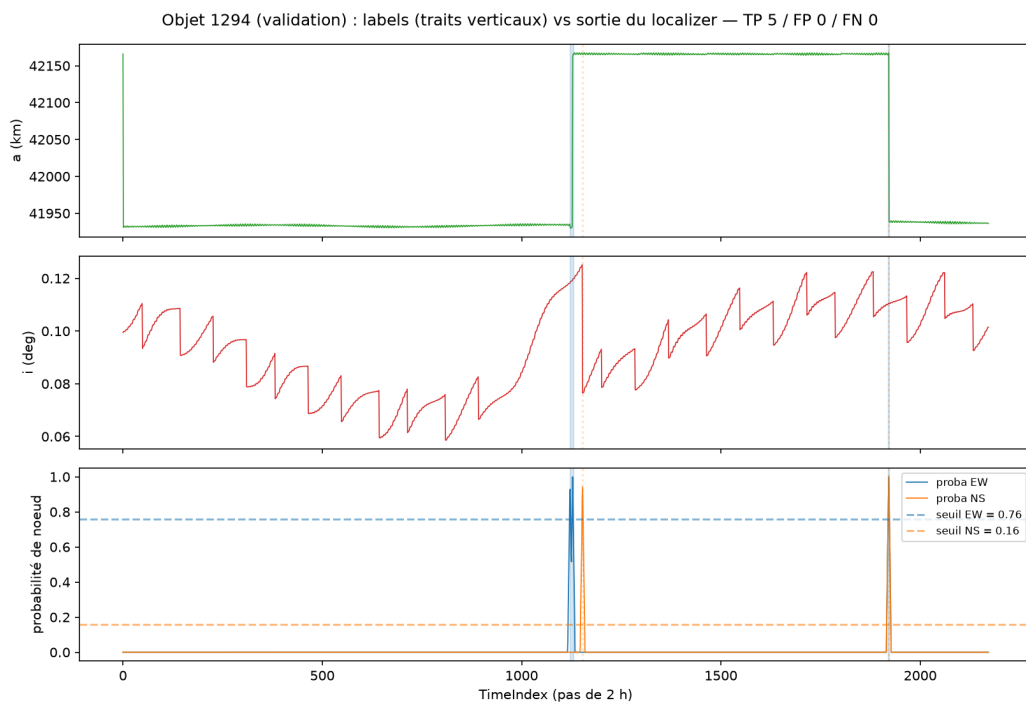
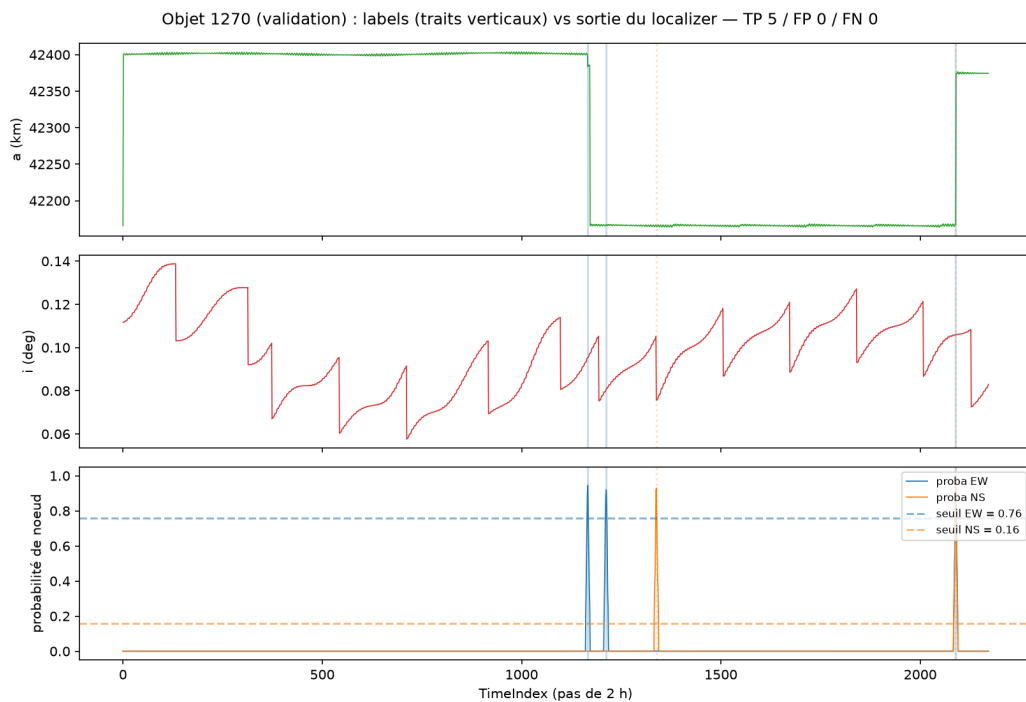


Objet 796 (validation) : labels (traits verticaux) vs sortie du localizer — TP 5 / FP 0 / FN 0



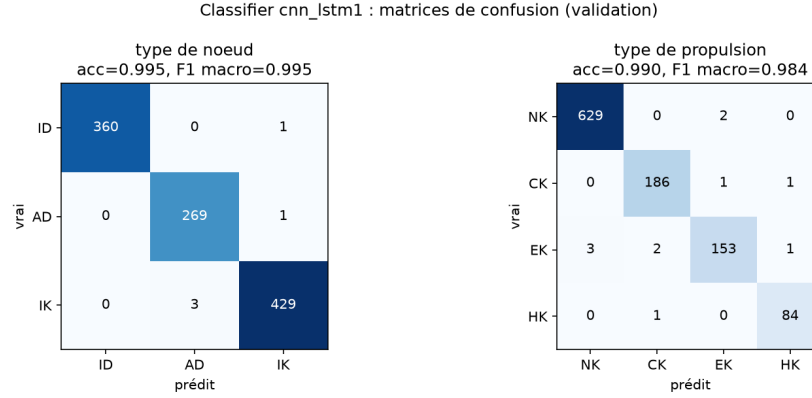
Objet 938 (validation) : labels (traits verticaux) vs sortie du localizer — TP 3 / FP 0 / FN 0





4.7 Classification des noeuds

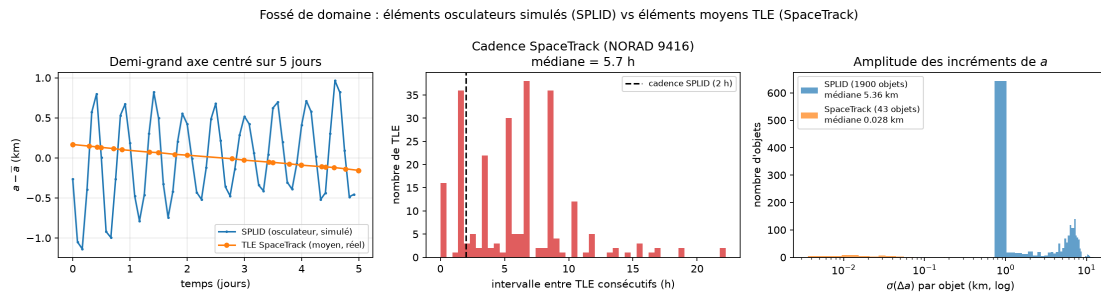
Le classifier `cnn_lstm1` sur les noeuds de validation :



La classification est quasi parfaite en validation : 99,5% d'exactitude sur le type de noeud (5 erreurs sur 1063) et 99,0% sur le type de propulsion. Les signatures des différents régimes (CK : impulsions fortes et espacées ; EK/HK : poussées faibles quasi continues) sont donc parfaitement séparables par le réseau — *lorsque la distribution d'entraînement couvre celle d'évaluation*, ce qui n'est plus le cas sur le jeu de test (section précédente).

5 Le problème du transfert vers SpaceTrack

L'objectif final reste la détection sur données réelles. Une passe d'inférence des modèles entraînés sur SPLID a donc été menée sur notre historique de TLE SpaceTrack de satellites GEO, remis au format SPLID (renommage des colonnes, ré-interpolation des éléments équinoxiaux sur une grille régulière de 2 h, découpage aux trous de données). **Les résultats sont inexploitable**, et l'analyse quantitative des deux domaines montre que ce n'est pas un problème d'architecture ni d'entraînement, mais d'incompatibilité de la nature même des données.



Éléments osculateurs contre éléments moyens. C'est le point central. La documentation du dataset l'indique explicitement : SPLID fournit les *éléments orbitaux osculateurs* de trajectoires produites par un simulateur haute fidélité. Le demi-grand axe y porte donc les oscillations à courte

période (12 h et 24 h en GEO, dues aux harmoniques du champ terrestre), d’amplitude crête-à-crête de plusieurs kilomètres — parfaitement visibles sur le panneau de gauche. Les TLE de SpaceTrack sont au contraire des *éléments moyens* au sens SGP4 : ces termes à courte période en ont été analytiquement retirés, et la série est lisse. Quantitativement, l’écart-type des incréments Δa entre échantillons successifs, qui est l’une des features principales du modèle, vaut en médiane :

$$\sigma(\Delta a)_{\text{SPLID}} = 5,36 \text{ km} \quad \text{contre} \quad \sigma(\Delta a)_{\text{SpaceTrack}} = 0,028 \text{ km},$$

soit un rapport de près de **200**. La composante dominante de la variance des features apprises n’existe tout simplement pas dans les TLE.

Cadence d’échantillonnage. SPLID est échantillonné exactement toutes les 2 h. Les TLE arrivent à intervalles irréguliers, de médiane 5,7 h pour le satellite GEO représenté, avec une queue au-delà de 20 h. La ré-interpolation sur une grille de 2 h fabrique donc de longues plages artificiellement lisses entre deux TLE, sans équivalent dans les données d’entraînement.

Données simulées contre données mesurées. Enfin, les trajectoires SPLID d’entraînement sont simulées, donc exemptes du bruit d’estimation propre aux TLE, qui résultent d’un ajustement sur observations radar et optiques.

Le modèle apprend donc des signatures fines dans des séries propres, régulières et riches en oscillations à courte période — signatures qui n’existent pas sous cette forme dans les TLE réels. C’est un *domain gap* au sens strict.

6 A suivre

- La pipeline SPLID est complète, reproductible et évaluée avec la métrique officielle du concours, sur le jeu d’entraînement comme sur le jeu de test : elle constitue un banc d’essai fiable pour itérer sur les architectures ;
- les performances en validation ($F_2 = 0,920$ en détection, classification des noeuds à 99%) montrent que l’approche par fenêtre glissante apprend effectivement les signatures de manœuvres, et que le seuil de détection y est réglable une fois pour toutes — deux propriétés que les méthodes par filtrage de Kalman n’offraient pas ;
- l’écart à notre score de test (0,675 contre 0,805 pour le gagnant) est à combler en priorité, par deux voies identifiées : une tête de classification dédiée aux noeuds SS (aujourd’hui prédits sur une fenêtre à moitié vide, et responsables de la majorité des faux positifs), et un rééquilibrage ou une augmentation des données ciblant les régimes de propulsion électrique et hybride, sous-représentés à l’entraînement ;
- la piste la plus prometteuse pour la détection out-of-plane est l’**extension du contexte temporel**, seul levier ayant produit un gain net (F_2 NS de 0,810 à 0,849) là où le changement d’architecture n’a rien donné. Les fenêtres au-delà de 128 pas, ou des convolutions dilatées à champ réceptif encore plus large, restent à explorer ;
- pour les données réelles, le transfert direct depuis SPLID est une impasse. Deux pistes : dégrader les données SPLID pour les rapprocher du domaine TLE (passage aux éléments moyens par filtrage des termes à courte période, sous-échantillonnage irrégulier, ajout de bruit d’estimation), et/ou entraîner directement sur des TLE réels labellisés (labels ILRS en LEO, labellisation manuelle en GEO, cf. rapport précédent) ;

- le pré-entraînement non supervisé (MAE) sur l'historique massif SpaceTrack, en cours, vise le même objectif : apprendre des représentations directement dans le domaine des TLE réels.